Proyecto 2 - Objectilang using Rascal

Integrantes:

- Carolina Ceballos Gómez - 202510267
- Juan Felipe Leon Barrero - 202510397

1. Correcciones del modelo anterior para Syntax.rsc:

Del modelo anterior realizamos algunas correcciones, en su mayoría de Typos. Por ejemplo, algunas reglas de producción tenían comillas tipográficas “”, estas no las reconoce Rascal, entonces fallan al pasarse al programa. Además, también tuvimos que corregir algunas reglas, pues generaban ambigüedad y tanto la sintaxis como el AST fallaban al no saber qué camino tomar.

El primer cambio que hicimos fue el NoTerminal inicial, lo dejamos como Module para seguir las convenciones de EBNF y los comentarios que nos hicieron en el proyecto 1. Es decir que ahora es:

Símbolo Distinguido/ Elemento Inicial (S0)

S_0 = **Module** (el no-terminal raíz del programa)

También definimos el layout (espacios, tabs y saltos de línea) como nos mostraba el tutorial para que Rascal los ignore entre tokens, y agrupamos las palabras reservadas en Reserved para que no se puedan utilizar en otro contexto.

A la regla Identifier le cambiamos la definición de Letter (Letter | Number)* a un identificador que empieza obligatoriamente con minúscula, que permite _ y no incluye las palabras reservadas. Ya que antes el identificador se podía comportar como un ClassName, siendo ambiguo el uso de estos.

La regla StringChar la reemplazamos por StringText , que acepta más caracteres y no es tan restrictiva, y por esto mismo eliminamos los no terminales Letter , UpperCase , LowerCase y Number , que ahora son clases de caracteres de Rascal.

También a la regla GenericSingleParam le agregamos la opción de pasarle un ClassName además de Identifier que ya estaba, pues algunos genéricos se pueden nombrar con mayúscula y si solo teníamos Identifier esto podría generar error.

A la regla Type la dividimos en dos, tipo y función , en vez de usar ("->" Type)? al final. Aunque acepta lo mismo, cada uno tiene su propio constructor en el AST. Asimismo, modificamos MethodDecl, el cuerpo (Block| Expression) lo convertimos en una regla aparte que se llama MethodBody .

También a la regla Statement le agregamos la opción de IfExpr , pues originalmente el if solo estaba en Expression y sus ramas then y else esperaban una Expression , lo que no nos dejaba usar println(...) dentro de ellas. Para eso creamos la regla IfBranch y ahora sus opciones son una Expression o un PrintStatement y también quitamos IfExpr de Primary para evitar ambigüedades y ahora el if solo es una sentencia.

Además, a Statement le quitamos la opción de Expression sola, porque era bastante ambiguo y Rascal ponía problema, ya que el lenguaje no usa ; para separar sentencias y una expresión puede empezar con - o (, entonces había varias formas de entender una sentencia y también se podían confundir con las llamadas. Entonces creamos CallStmt que ahora solo acepta como sentencia una llamada que empieza con un nombre.

Los booleanos o las expresiones de este tipo fueron nuestro mayor problema pues había varios caminos diferentes para llegar a ellos y Rascal no elegía, pues teníamos en la regla de Comparison que se revisaban dos Expression pero BoolAtomico incluía "(" BoolExpr ")" , true y false . Así que tuvimos que hacer que Expression fuera BoolExpr, y dejar los paréntesis solo en Primary y true/false solo en BoolLiteral.

También tuvimos que utilizar recursión por izquierda para definir el orden de precedencia y el nivel jerárquico de cada regla, por ejemplo, para evitar ambigüedades, SumExpr debe ir antes que MulExp y esta debe ir antes de PowExpr. Además, en PostFix eliminamos ArgList y simplemente usamos la lista de expresiones separadas por coma.

Ahora Expression es BoolExpr, y la precedencia quedó primero or, luego and, luego neg, luego las comparaciones, luego suma o resta, y así hacia abajo. Comparison ahora compara SumExpr (no Expression), lo que evita la ambigüedad.

Y finalmente, las listas X ("," X)* pasaron a {X ","}+ o {X ","}* , porque es la forma de Rascal para listas con separador y nos facilitó la construcción del AST. Para los símbolos <, >, -> y <- los escribimos separados con \ ya que Rascal los tiene ya con un significado y sin esto la gramática no compila. Tuvimos que investigar un poco para saber cuál era este error. También le pusimos un constructor a cada producción para construir el AST.

2. Algunas aclaraciones extras del proyecto

Para facilitar un poco la calificación de este proyecto, queremos hacer algunas aclaraciones sobre las justificaciones de la construcción de nuestro proyecto en Rascal. Entre estas están cómo hicimos el highlighting, algunas pruebas que hicimos, los archivos extra que construimos con ayuda de lo que aprendimos en el tutorial para optimizar estas pruebas, etc.

La primera es sobre el highlighting, que es pedida para el punto 1: tenemos el archivo Plugin.rsc funcional y en el archivo Syntax.rsc tenemos algunas categorías (@Category) para este highlighting. Tenemos las categorías de string, number, type que colorean a la perfección. Tenemos también la categoría comment (la utilizamos para los identificadores). Esta pinta de un color gris que se puede confundir con el blanco (es decir, confundir como si

no estuviera coloreada), pero sí lo está, solo que es difícil a simple vista notar la diferencia entre estos colores.

También, mientras íbamos construyendo la sintaxis y el AST, creamos unas pruebas para intentar probar todos los casos que debían funcionar, parsear, etc. Todo eran archivos .obj y este es un ejemplo:

```
object Punto(val x: Int, var y: Int) {
  val z = 42
}

method main {
  val nombre = "hola"
  println("Hola $nombre")
}
```

Eran varios archivos que probaban diferentes casos. Decidimos no conservarlos y no dejarlos en la entrega por recomendación de la monitora que nos comentó que igual al calificar se van a hacer tests, así que solo fueron verificaciones para revisar que todo estuviera funcionando.

Con ayuda de lo que vimos en el tutorial, también creamos otros archivos para facilitar estas pruebas, el archivo Parser.rsc que recibe el programa como texto y lo convierte en un árbol de parseo usando la gramática de Syntax.rsc. Y el archivo Imploder.rsc convierte ese árbol en el AST, como explica el tutorial. Para que esto funcione, el AST debe tener un constructor por cada producción de la gramática y por eso lo mencionamos en los cambios realizados. Sabemos que no eran entregables explícitos del proyecto, pero como aprendimos a hacerlo en el tutorial, quisimos agregarlos y creemos que tal vez podría facilitar la revisión de nuestro trabajo.

3. Uso de Soluciones Previas del Proyecto 1

Dado que nuestro grupo sigue estando conformado por exactamente las mismas personas que en el Proyecto 1, la gramática que utilizamos en la solución presentada es de nuestra autoría original y no proviene de ningún otro grupo.